

7. Problems associated with the OS/environment

How to identify the problem

Typical problems that occur in a running Java program are: the program does not respond, aborts abnormally, displays exceptions or error messages in the console/GUI, the GUI freezes or grays out, the program crashes and a core dump occurs, or any other unexpected/abnormal behaviour.

In such cases, debugging and finding out the reason and solution for the problem is a tough task. In general, the coder has to write the program/application in a way that either such problems can be bypassed, or each activity (milestone) with all possible information is recorded during its runtime so that it can help in identifying the cause of the current problem.

This can be done with the help of the following:

1. Log files
2. Stack trace, dump trace output
3. Logging information in the database during each stage
4. Console output
5. Message boards and error messages
6. Dialogue boxes and information windows showing the current status

Log files

What's most useful when diagnosing any type of Java problem is the information that we get from log files, which serves as first hand details for identifying the problem at hand.

Keeping this in mind, code should be written in such a way that the log files give the maximum possible details. This is how log files become the main source in helping anyone to understand the execution flow.

The log files can give details like the method flow, values used and exceptions occurred during execution. Based on the type of display, logging can be classified as follows.

1. **Console logging:** This will print the log in the console, where the Java program is started. If the program is a GUI that runs from the browser, then the browser console can be used to print the log details. In non-Windows platforms like UNIX, we can start the program in the background so that when we print the console logs in these platforms, we cannot see them. Hence, in these platforms, it is better to redirect the program's console output (logging) to a file itself.
2. **File logging:** We can do exhaustive logging in files by directly using streams in Java code or using the facility from tools like log4j (<http://logging.apache.org/log4j/docs/>).

In the Java language, where a preprocessor is not available, log statements increase the size of the code and reduce its speed, even when logging is turned off. Given that a reasonably sized application may contain thousands

of log statements, speed is of particular importance.

Hence, we have to make the logging configurable or have logging with categories like the following:

- An INFO log, which gives the general method flow, entry/exit of the method, branching, etc.
- A DEBUG log, which gives the values for objects, object creation details, etc.
- A WARNING log, where we bypass minor errors like taking the default value when the given value does not satisfy the boundary conditions, the exception to which will not affect the executing flow, etc.
- An ERROR log, which gives the fatal errors during execution. Fatal errors for a Java program can be environmental errors, communication errors, database logging errors, IO errors, device handling problems, etc.

We can make logging configurable to enable/disable it at runtime without modifying the application binary. Logging behaviour can be controlled by editing a configuration file, without touching the application binary for this behaviour.

Logging equips the developer with a detailed context for application failures. On the other hand, testing provides quality assurance and confidence in the application. Logging and testing should not be mistaken for each other. They are complementary. When logging is wisely used, it can prove to be an essential tool.

Using Stacktrace to debug code

While coding or debugging, we can make use of Stacktrace to analyse problems. While coding, we can do this using the `printStackTrace()` method as follows:

```
try
{
    <do something>
}
catch(Exception ex)
{
    ex.printStackTrace();
}
```

When we run the above code, we will get a trace output from the stack that looks somewhat like what follows:

```
java.lang.Exception
at Excep1.main(Excep1.java:57)
    at sun.rmi.server.UnicastServerRef.dispatch(Unknown
Source)
        at sun.rmi.transport.Transport.serviceCall(Unknown
Source)
            at sun.rmi.transport.tcp.TCPTransport.
handleMessages(Unknown Source)
                at sun.rmi.transport.tcp.TCPTransport$ConnectionHandler.
run(Unknown Source)
                    at java.lang.Thread.run(Unknown Source)
```